



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

CSIT327-[SECTION] Information Management 2

Software Design Document (SDD)

Project Title: CashGuard: A Web-Based Personal Finance Management System for College Students and Campus Employees

Prepared By: Java, Zorel Adrean R.

Version: 1.0

Date: October 9, 2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	Oct 9, 2026	Zorel Adrean R. Java	Initial draft: system description	Draft
0.2	Oct 9, 2026	Zorel Adrean R. Java	Added feature list	Review
0.3	Oct 9, 2026	Zorel Adrean R. Java	Added business rules	Review
0.4	Oct 9, 2026	Zorel Adrean R. Java	Added data model (ERD and data dictionary)	Review
1.0	Oct 9, 2026	Zorel Adrean R. Java	Version submitted for approval	Final

TABLE OF CONTENTS

Contents

1.0 DESCRIPTION OF THE PROPOSED SYSTEM.....	4
2.0 FEATURE LIST.....	7
3.0 BUSINESS RULES.....	10
4.0 DATA MODELLING.....	13

1.0 DESCRIPTION OF THE PROPOSED SYSTEM

1.1 Project Overview

CashGuard is a web-based personal finance management system for college students and campus employees. Users record their expenses and income, plan a monthly budget divided by spending category, and track savings goals through individual deposits and withdrawals. The system turns these records into reports such as budget versus actual spending per category, monthly income versus expenses, and savings goal progress. Administrators manage user accounts and the default expense categories, and view campus-wide spending summaries. CashGuard is built with Python and Django, and all data is stored in a PostgreSQL relational database.

1.2 Project Information

Project Name: CashGuard

Domain: Personal Finance / Education

Technology: Python, Django (full-stack web), PostgreSQL

Primary Users:

1. Students: record their allowance, expenses, monthly budgets, and savings goals, and view their own reports
2. Employees (campus staff and faculty): use the same personal finance features as students
3. Administrators: manage user accounts and default categories, monitor login activity, and view system-wide reports

1.3 Problem Statement

Many college students and campus employees manage their money with a mix of memory, paper notes, spreadsheets, and their e-wallet transaction history. Students in particular live on a fixed weekly or monthly allowance and often run out of money before the next one arrives, without knowing where the money went. Spending is spread across cash, GCash or Maya, and cards, so there is no single place where all of it is recorded.

Notebooks and spreadsheets do not check the data: amounts are mistyped, the same expense is written twice, a budget is set for a month but never compared with actual spending, and savings goals are tracked as a single number that is overwritten instead of a history of deposits. Producing a summary such as "how much did I spend on food this month compared with my budget" requires manual computation, so it is rarely done. The first version of CashGuard (PHP/MySQL)

stored budgets and goals, but it did not tie expenses to a budget month, allowed duplicate budgets, and never updated the amount saved toward a goal.

1.4 Proposed Solution

CashGuard keeps every expense, income entry, budget, and savings transaction of a user in one centralized PostgreSQL database. Every expense belongs to a category, every budget belongs to a month and is divided into category allocations, and every savings goal keeps a ledger of contributions. Database constraints reject invalid data (negative amounts, duplicate budgets for the same month, invalid dates), and the remaining budget and amount saved are computed from the stored records instead of being typed in, so they are always consistent. Users search and filter their records and view reports that join several tables, while administrators see aggregated campus-wide summaries.

1.5 Objectives

1. Design a normalized PostgreSQL database of at least six related tables that stores users, student profiles, categories, expenses, incomes, budgets, budget allocations, savings goals, and goal contributions without redundancy
2. Enforce business rules through database constraints (primary keys, foreign keys, UNIQUE, CHECK, NOT NULL) and Django validation
3. Allow users to create, view, update, and delete their expenses, incomes, budgets, savings goals, and goal contributions
4. Allow users to search and filter records by keyword, category, payment method, type, and date range
5. Generate reports from multiple tables, including budget versus actual spending per category, monthly income versus expenses, and savings goal progress
6. Provide role-based access for students, employees, and administrators, including account management and login lockout after repeated failed attempts

1.6 Scope

Included Features:

- User registration, login, logout, and password change with role-based access (Student, Employee, Admin)
- Student profile (program and year level)
- Expense management (CRUD) with search and filtering

- Income management (CRUD) with search and filtering
- Expense category management (system default categories and user-defined categories)
- Monthly budgets with per-category allocations
- Savings goals with a deposit and withdrawal history
- Personal dashboard and reports (budget vs. actual, income vs. expenses, goal progress)
- Admin user management, default category management, login activity log, and system-wide reports

Excluded Features:

- Automatic import from banks, GCash, Maya, or other e-wallets
- Online payments or money transfers between users
- Email, SMS, and push notifications
- Mobile app and offline mode
- AI-generated financial advice
- Multi-currency support

1.7 Limitations

- All amounts are in Philippine pesos (PHP); there is no currency conversion
- All records are entered manually by the user; the system does not verify them against real accounts
- One budget per user per calendar month; weekly or custom-period budgets are not supported
- The system requires an internet connection and a web browser; English only
- Administrators can view aggregated reports and records for monitoring but cannot edit a user's financial records

2.0 FEATURE LIST

2.1 Feature List (MoSCoW)

MUST HAVE

1. User authentication and role-based access (register, login, logout)
2. Expense management (add, view, edit, delete) with search and filters
3. Category management (default and user-defined categories)
4. Monthly budgets with category allocations
5. Savings goals and goal contributions
6. Personal reports (budget vs. actual, goal progress)
7. Admin user management

SHOULD HAVE

1. Income tracking and monthly income vs. expense summary
2. Admin system-wide reports (spending per month, per category, per account type and program)
3. Login attempt log with temporary account lockout
4. Profile update and password change

COULD HAVE

1. Export of reports to CSV
2. Charts on the dashboards (Chart.js)
3. Overspending warning when a category exceeds its allocation

WON'T HAVE

1. Bank and e-wallet integration
2. Email/SMS notifications
3. Mobile app and offline mode

2.2 Feature Descriptions

ID	Feature	Description	User	Tables Used
F-01	User Registration and Authentication	Users register with a unique username, name, gender, and account type (Student or Employee); students also enter program and year level. Users log in and log out; admins are routed to the admin dashboard.	Student, Employee, Admin	users, student_profiles, login_attempts

ID	Feature	Description	User	Tables Used
F-02	Profile and Password Management	Users view and update their name and student details and change their password after confirming the current one.	Student, Employee	users, student_profiles
F-03	Expense Management	Users add, view, edit, and delete expenses (description, category, amount, date, payment method). The list can be searched by description and filtered by category, payment method, and date range.	Student, Employee	expenses, categories
F-04	Category Management	Admins maintain the default categories (e.g., Food, Transportation, School Supplies). Users add, rename, and deactivate their own categories. Categories in use are deactivated, not deleted.	Admin, Student, Employee	categories
F-05	Income Management	Users add, view, edit, and delete income entries (allowance, salary, scholarship, etc.) and filter them by type and date range.	Student, Employee	incomes
F-06	Monthly Budget Planning	Users create one budget per month, divide it into category allocations, and edit or delete it. The remaining budget is computed from the expenses of that month.	Student, Employee	budgets, budget_allocations, categories, expenses
F-07	Savings Goals	Users create, edit, cancel, and delete savings goals with a target amount and target date, and filter goals by status.	Student, Employee	savings_goals
F-08	Goal Contributions	Users record deposits and withdrawals for a goal. The amount saved is the sum of deposits minus withdrawals; a goal is marked Completed when it reaches its target.	Student, Employee	goal_contributions, savings_goals
F-09	Personal Dashboard	Shows the current month's budget, total spent, remaining budget, total income, and the progress of active goals.	Student, Employee	budgets, budget_allocations, expenses, incomes, savings_goals, goal_contributions
F-10	Personal Reports	Budget vs. actual spending per category for a selected month; monthly income vs. expense summary for a year; savings goal progress report.	Student, Employee	budgets, budget_allocations, categories, expenses, incomes, savings_goals, goal_contributions

ID	Feature	Description	User	Tables Used
F-11	Admin User Management	Admins search users by name or username, filter by account type and program, view details, deactivate or reactivate accounts, and delete accounts.	Admin	users, student_profiles
F-12	Admin System-Wide Reports	Total users per account type and gender, total spending per month and year, spending per category, average monthly budget, average spending per program, and goal completion rate.	Admin	users, student_profiles, expenses, categories, budgets, savings_goals, goal_contributions
F-13	Login Activity Monitor	Every login attempt is logged. Accounts are locked temporarily after repeated failures; admins can view and filter the log.	Admin	login_attempts, users

3.0 BUSINESS RULES

3.1 Business Rules

Rule ID	Business Rule	Related Entity	Enforcement
BR-01	Each user must have a unique username.	users	UNIQUE (username), NOT NULL
BR-02	A user's account type must be STUDENT, EMPLOYEE, or ADMIN.	users	CHECK (account_type IN ('STUDENT','EMPLOYEE','ADMIN'))
BR-03	Public registration may create only STUDENT or EMPLOYEE accounts; ADMIN accounts are created only by an existing admin.	users	Application logic (registration form choices, admin-only view)
BR-04	A password must be at least 8 characters long and is stored only as a hash, never as plain text.	users	Application logic (Django password validators and hashing); NOT NULL
BR-05	A user's gender must be MALE, FEMALE, or OTHER.	users	CHECK (gender IN ('MALE','FEMALE','OTHER'))
BR-06	Every STUDENT account has exactly one student profile; EMPLOYEE and ADMIN accounts have none.	student_profiles	FK user_id with UNIQUE (user_id); application logic (created in the same transaction as the user)
BR-07	A student's year level must be from 1 to 4, and the program must be one of the offered programs.	student_profiles	CHECK (year_level BETWEEN 1 AND 4); CHECK (program IN ('BSIT','BSCS','BSIS','BSEMC'))
BR-08	Category names are unique per user, and default (system) category names are unique among themselves.	categories	UNIQUE NULLS NOT DISTINCT (user_id, name)
BR-09	A category used by any expense or budget allocation cannot be deleted; it can only be deactivated.	categories, expenses, budget_allocations	FK ON DELETE NO ACTION, DEFERRABLE INITIALLY DEFERRED (Django on_delete=PROTECT); is_active column
BR-10	A user may only use active default categories or their own active categories.	categories, expenses	Application logic (form queryset filter)
BR-11	Every expense must belong to exactly one user and one category.	expenses	FK user_id NOT NULL; FK category_id NOT NULL
BR-12	An expense amount must be greater than zero.	expenses	CHECK (amount > 0)
BR-13	An expense or income date cannot be later than the current date.	expenses, incomes	Application logic (form validation)
BR-14	An expense's payment method must be CASH, GCASH, MAYA, CARD, BANK, or OTHER.	expenses	CHECK (payment_method IN (...)); DEFAULT 'CASH'

Rule ID	Business Rule	Related Entity	Enforcement
BR-15	An income amount must be greater than zero, and its type must be ALLOWANCE, SALARY, SCHOLARSHIP, PART_TIME, GIFT, or OTHER.	incomes	CHECK (amount > 0); CHECK (income_type IN (...))
BR-16	A user may have at most one budget for the same month and year.	budgets	UNIQUE (user_id, budget_month, budget_year)
BR-17	A budget month must be from 1 to 12, the year from 2000 to 2100, and the total amount greater than zero.	budgets	CHECK (budget_month BETWEEN 1 AND 12); CHECK (budget_year BETWEEN 2000 AND 2100); CHECK (total_amount > 0)
BR-18	A category may be allocated only once in a budget, and each allocation must be greater than zero.	budget_allocations	UNIQUE (budget_id, category_id); CHECK (allocated_amount > 0)
BR-19	The sum of a budget's category allocations cannot exceed the budget's total amount.	budget_allocations, budgets	Application logic (validated in a transaction on save)
BR-20	The amount spent and remaining budget are computed from the expenses dated within the budget's month and year; they are never stored.	budgets, expenses	Application logic (report query); no stored spent column
BR-21	A savings goal's target amount must be greater than zero, and its target date must be after its start date.	savings_goals	CHECK (target_amount > 0); CHECK (target_date > start_date)
BR-22	A user cannot have two savings goals with the same name.	savings_goals	UNIQUE (user_id, goal_name)
BR-23	A goal's status must be ACTIVE, COMPLETED, or CANCELLED.	savings_goals	CHECK (status IN ('ACTIVE','COMPLETED','CANCELLED')); DEFAULT 'ACTIVE'
BR-24	A contribution amount must be greater than zero, and its type must be DEPOSIT or WITHDRAWAL.	goal_contributions	CHECK (amount > 0); CHECK (contribution_type IN ('DEPOSIT','WITHDRAWAL'))
BR-25	Contributions may be added only to ACTIVE goals.	goal_contributions, savings_goals	Application logic
BR-26	A withdrawal cannot be greater than the goal's current amount saved (deposits minus withdrawals).	goal_contributions	Application logic (validated in a transaction)
BR-27	A goal is set to COMPLETED when its amount saved reaches or exceeds its target amount.	savings_goals, goal_contributions	Application logic (after each contribution is saved)
BR-28	After 3 consecutive failed login attempts for a username, login for that username is blocked for 2 minutes.	login_attempts	Application logic using login_attempts records

Rule ID	Business Rule	Related Entity	Enforcement
BR-29	A deactivated user (is_active = FALSE) cannot log in.	users	Application logic (authentication backend); NOT NULL DEFAULT TRUE
BR-30	Students and employees can view and change only their own records; admins can view all records but cannot edit another user's financial records.	all user-owned tables	Application logic (queriesets filtered by user_id; role checks)
BR-31	Deleting a user deletes the user's profile, categories, expenses, incomes, budgets, goals, and contributions; their login attempts are kept with no user link.	all user-owned tables, login_attempts	FK ON DELETE CASCADE; login_attempts.user_id ON DELETE SET NULL

4.0 DATA MODELLING

4.1 Entity Relationship Diagram

Crow's foot notation. PK = primary key, FK = foreign key, UK = unique. The diagram is shown on the next page in landscape for readability; the full constraints of every column are listed in the data dictionary (Section 4.4).

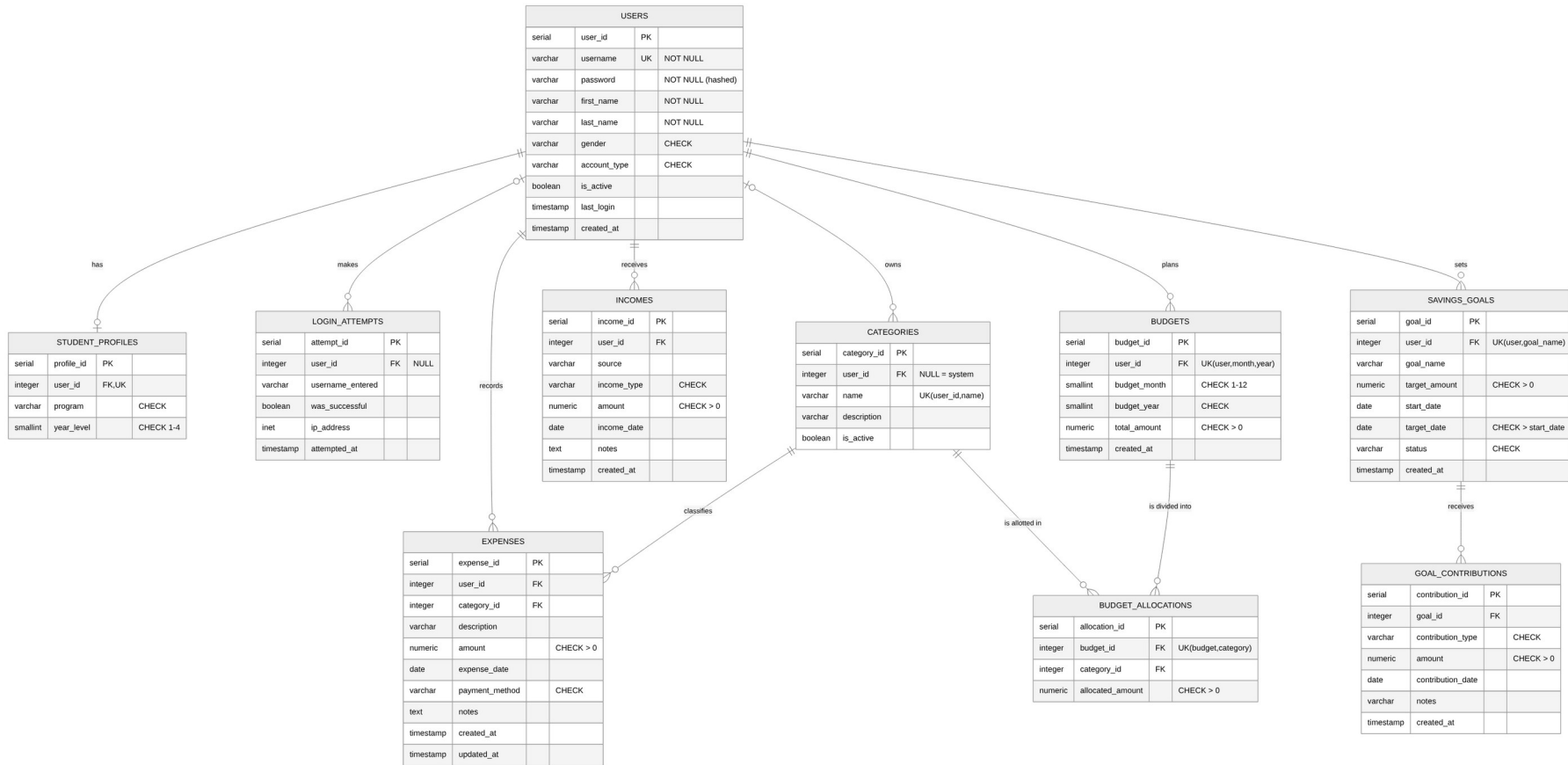


Figure 1. CashGuard Entity Relationship Diagram

4.2 Entities

Entity	Description
users	Registered students, employees, and administrators (Django custom user model)
student_profiles	Program and year level of users whose account type is STUDENT
login_attempts	Log of every successful and failed login attempt, used for lockout and monitoring
categories	Expense categories, either system defaults (no owner) or created by a user
expenses	Money spent by a user, with category, amount, date, and payment method
incomes	Money received by a user, such as allowance, salary, or scholarship
budgets	A user's spending limit for one calendar month
budget_allocations	The part of a monthly budget assigned to a category
savings_goals	A target amount a user wants to save by a target date
goal_contributions	Deposits to and withdrawals from a savings goal

4.3 Relationships

Relationship	Cardinality	Description
users – student_profiles	1 : 0..1	A student user has exactly one profile; employees and admins have none. Each profile belongs to one user.
users – login_attempts	0..1 : M	A user can have many login attempts; an attempt with an unknown username has no user.
users – categories	0..1 : M	A user can create many categories; a default category has no owner.
users – expenses	1 : M	A user can record many expenses; each expense belongs to one user.
users – incomes	1 : M	A user can record many incomes; each income belongs to one user.
users – budgets	1 : M	A user can have many monthly budgets (one per month); each budget belongs to one user.
users – savings_goals	1 : M	A user can have many savings goals; each goal belongs to one user.
categories – expenses	1 : M	A category classifies many expenses; each expense has one category.
budgets – budget_allocations	1 : M	A budget is divided into many allocations; each allocation belongs to one budget.
categories – budget_allocations	1 : M	A category can be allocated in many budgets; each allocation is for one category.
savings_goals – goal_contributions	1 : M	A goal can have many contributions; each contribution belongs to one goal.

4.4 Data Dictionary

Note on Django tables: users is implemented as a custom user model (AUTH_USER_MODEL) and replaces Django's auth_user table. Django's built-in framework tables (auth_group, auth_permission, django_session, django_content_type, django_migrations, django_admin_log) are not part of this data model. The 10 tables above are the system's own tables.

Table: users

Column Name	Data Type	Constraints	Description
user_id	SERIAL	PK	Unique identifier of the user
username	VARCHAR(50)	NOT NULL, UNIQUE	Login name
password	VARCHAR(128)	NOT NULL	Hashed password (Django PBKDF2)
first_name	VARCHAR(50)	NOT NULL	First name
last_name	VARCHAR(50)	NOT NULL	Last name
gender	VARCHAR(10)	NOT NULL, CHECK (gender IN ('MALE','FEMALE','OTHER'))	Gender of the user
account_type	VARCHAR(10)	NOT NULL, DEFAULT 'STUDENT', CHECK (account_type IN ('STUDENT','EMPLOYEE','ADMIN'))	Role / access level
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	FALSE if the account is deactivated
last_login	TIMESTAMP	NULL	Date and time of the last successful login
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the account was created

Table: student_profiles

Column Name	Data Type	Constraints	Description
profile_id	SERIAL	PK	Unique identifier of the profile
user_id	INTEGER	NOT NULL, UNIQUE, FK → users(user_id) ON DELETE CASCADE	Student who owns the profile
program	VARCHAR(10)	NOT NULL, CHECK (program IN ('BSIT','BSCS','BSIS','BSEMC'))	Degree program
year_level	SMALLINT	NOT NULL, CHECK (year_level BETWEEN 1 AND 4)	Current year level

Table: login_attempts

Column Name	Data Type	Constraints	Description
attempt_id	SERIAL	PK	Unique identifier of the attempt
user_id	INTEGER	NULL, FK → users(user_id) ON DELETE SET NULL	Matching user, if the username exists
username_entered	VARCHAR(50)	NOT NULL	Username typed on the login form
was_successful	BOOLEAN	NOT NULL	TRUE if the login succeeded
ip_address	INET	NULL	IP address of the client
attempted_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time of the attempt

Table: categories

Column Name	Data Type	Constraints	Description
category_id	SERIAL	PK	Unique identifier of the category
user_id	INTEGER	NULL, FK → users(user_id) ON DELETE CASCADE	Owner; NULL for a default (system) category
name	VARCHAR(50)	NOT NULL	Category name (e.g., Food)
description	VARCHAR(255)	NULL	Short description
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	FALSE if the category is hidden from new entries

Table constraint: UNIQUE NULLS NOT DISTINCT (user_id, name)

Table: expenses

Column Name	Data Type	Constraints	Description
expense_id	SERIAL	PK	Unique identifier of the expense
user_id	INTEGER	NOT NULL, FK → users(user_id) ON DELETE CASCADE	User who spent the money
category_id	INTEGER	NOT NULL, FK → categories(category_id) ON DELETE NO ACTION DEFERRABLE INITIALLY DEFERRED	Category of the expense
description	VARCHAR(255)	NOT NULL	What the money was spent on
amount	NUMERIC(12,2)	NOT NULL, CHECK (amount > 0)	Amount spent in pesos

Column Name	Data Type	Constraints	Description
expense_date	DATE	NOT NULL	Date the expense happened
payment_method	VARCHAR(10)	NOT NULL, DEFAULT 'CASH', CHECK (payment_method IN ('CASH','GCASH','MAYA','CARD','BANK','OTHER'))	How it was paid
notes	TEXT	NULL	Optional remarks
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the record was created
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the record was last edited

Table: incomes

Column Name	Data Type	Constraints	Description
income_id	SERIAL	PK	Unique identifier of the income
user_id	INTEGER	NOT NULL, FK → users(user_id) ON DELETE CASCADE	User who received the money
source	VARCHAR(100)	NOT NULL	Where the money came from (e.g., Parents)
income_type	VARCHAR(12)	NOT NULL, CHECK (income_type IN ('ALLOWANCE','SALARY','SCHOLARSHIP','PART_TIME','GIFT','OTHER'))	Type of income
amount	NUMERIC(12,2)	NOT NULL, CHECK (amount > 0)	Amount received in pesos
income_date	DATE	NOT NULL	Date the money was received
notes	TEXT	NULL	Optional remarks
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the record was created

Table: budgets

Column Name	Data Type	Constraints	Description
budget_id	SERIAL	PK	Unique identifier of the budget
user_id	INTEGER	NOT NULL, FK → users(user_id) ON DELETE CASCADE	Owner of the budget
budget_month	SMALLINT	NOT NULL, CHECK (budget_month BETWEEN 1 AND 12)	Month the budget covers

Column Name	Data Type	Constraints	Description
budget_year	SMALLINT	NOT NULL, CHECK (budget_year BETWEEN 2000 AND 2100)	Year the budget covers
total_amount	NUMERIC(12,2)	NOT NULL, CHECK (total_amount > 0)	Spending limit for the month
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the budget was created

Table constraint: UNIQUE (user_id, budget_month, budget_year)

Table: budget_allocations

Column Name	Data Type	Constraints	Description
allocation_id	SERIAL	PK	Unique identifier of the allocation
budget_id	INTEGER	NOT NULL, FK → budgets(budget_id) ON DELETE CASCADE	Budget being divided
category_id	INTEGER	NOT NULL, FK → categories(category_id) ON DELETE NO ACTION DEFERRABLE INITIALLY DEFERRED	Category receiving the allocation
allocated_amount	NUMERIC(12,2)	NOT NULL, CHECK (allocated_amount > 0)	Limit for this category in the month

Table constraint: UNIQUE (budget_id, category_id)

Table: savings_goals

Column Name	Data Type	Constraints	Description
goal_id	SERIAL	PK	Unique identifier of the goal
user_id	INTEGER	NOT NULL, FK → users(user_id) ON DELETE CASCADE	Owner of the goal
goal_name	VARCHAR(100)	NOT NULL	Name of the goal (e.g., New Laptop)
target_amount	NUMERIC(12,2)	NOT NULL, CHECK (target_amount > 0)	Amount the user wants to save
start_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Date saving started
target_date	DATE	NOT NULL	Date the goal should be reached
status	VARCHAR(10)	NOT NULL, DEFAULT 'ACTIVE', CHECK (status IN ('ACTIVE','COMPLETED','CANCELLED'))	Current status of the goal
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the goal was created

Table constraints: UNIQUE (user_id, goal_name); CHECK (target_date > start_date)

Table: goal_contributions

Column Name	Data Type	Constraints	Description
contribution_id	SERIAL	PK	Unique identifier of the contribution
goal_id	INTEGER	NOT NULL, FK → savings_goals(goal_id) ON DELETE CASCADE	Goal the money is added to or taken from
contribution_type	VARCHAR(10)	NOT NULL, DEFAULT 'DEPOSIT', CHECK (contribution_type IN ('DEPOSIT','WITHDRAWAL'))	Deposit or withdrawal
amount	NUMERIC(12,2)	NOT NULL, CHECK (amount > 0)	Amount in pesos
contribution_date	DATE	NOT NULL	Date of the deposit or withdrawal
notes	VARCHAR(255)	NULL	Optional remarks
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time the record was created

4.5 Sample Report Queries

The reports in F-10 and F-12 are computed from several tables. The main queries (PostgreSQL, run through the Django ORM or raw SQL) are:

Budget vs. actual per category (F-10) — uses budgets + budget_allocations + categories + expenses

```
SELECT c.name, ba.allocated_amount, COALESCE(SUM(e.amount),0) AS spent,
       ba.allocated_amount - COALESCE(SUM(e.amount),0) AS remaining
FROM budgets b
JOIN budget_allocations ba ON ba.budget_id = b.budget_id
JOIN categories c ON c.category_id = ba.category_id
LEFT JOIN expenses e ON e.category_id = c.category_id AND e.user_id = b.user_id
  AND EXTRACT(MONTH FROM e.expense_date) = b.budget_month
  AND EXTRACT(YEAR FROM e.expense_date) = b.budget_year
WHERE b.user_id = %s AND b.budget_month = %s AND b.budget_year = %s
GROUP BY c.name, ba.allocated_amount ORDER BY spent DESC;
```

Savings goal progress (F-10) — uses savings_goals + goal_contributions

```
SELECT g.goal_name, g.target_amount, g.target_date, g.status,
       COALESCE(SUM(CASE WHEN gc.contribution_type = 'DEPOSIT' THEN gc.amount
                        ELSE -gc.amount END), 0) AS amount_saved
FROM savings_goals g
LEFT JOIN goal_contributions gc ON gc.goal_id = g.goal_id
WHERE g.user_id = %s
GROUP BY g.goal_id ORDER BY g.target_date;
```

Spending per month and account type (F-12) — uses expenses + users + categories

```
SELECT DATE_TRUNC('month', e.expense_date) AS month, u.account_type,
       c.name AS category, SUM(e.amount) AS total_spent, COUNT(*) AS entries
FROM expenses e
```

```
JOIN users u ON u.user_id = e.user_id  
JOIN categories c ON c.category_id = e.category_id  
GROUP BY 1, 2, 3 ORDER BY 1, 2, total_spent DESC;
```